When Minutes Mean Lives

Optimizing Ambulance Placement in New York City with Equity Constraints and Reinforcement Learning

By Hilmi

HILMI

Research Areas: Operations Research · Machine Learning · Health Equity · Urban Analytics

Key Methods: Facility Location (p Median, p Center) · Proximal Policy Optimization · Graph Neural Networks

Datasets: NYC 911 EMS Dispatch Data (18M+ incidents) · OpenStreetMap Road Networks

Platform: Kaggle (GPU T4) · Python · PuLP · Stable Baselines3 · OSMnx · SimPy

Project Statement

This project sits at the intersection of **operations research**, **machine learning**, and **social impact**. I am driven by a question that is both technical and deeply human: *Can algorithms decide where to place ambulances so that response times are fast and fair, ensuring no neighborhood is left behind?*

Emergency medical services represent one of the most direct connections between an algorithm's output and a human outcome. When an ambulance arrives two minutes faster because a station was placed in a better location, that difference can separate survival from death for a cardiac arrest patient. This is not a hypothetical scenario. Research by Holmén et al. (2020) established that survival from out of hospital cardiac arrest decreases by 7 to 10 percent for every additional minute of delay.

What makes this project more than a standard optimization exercise is the **equity dimension**. During the early exploration of NYC's EMS data, I noticed a pattern that troubled me: response times in the Bronx and parts of eastern Brooklyn were consistently worse than in Manhattan and Staten Island. These communities tend to be lower income and disproportionately non white. This realization transformed the project from a technical exercise into something with genuine policy relevance.

The technical challenge is substantial. It requires integrating real road network data from OpenStreetMap, solving mixed integer programs for facility location, training reinforcement learning agents for dynamic ambulance redeployment, and evaluating everything through simulation. This breadth is deliberate. I wanted to demonstrate that these techniques are not isolated academic tools but components of a unified pipeline that can address a single, important question.

What I learned. Perhaps the most valuable outcome was discovering, through careful numerical auditing, that my initial equity constrained formulation produced results that were *worse on both efficiency and equity* than the baseline. Finding and fixing this bug required deeper engagement with the mathematics than simply "making the code run." The Gini coefficient linearization, while valid in theory, created degenerate solutions in practice. This experience reinforced my belief that **reproducible research requires more than reproducible code**: it requires someone who understands the theory well enough to know when the numbers are wrong.

Broader research trajectory. This project reflects my systematic approach to research: I begin with a real problem (saving lives), connect it to mathematical structure (the facility location polytope, Rawlsian justice, Markov decision processes), implement state of the art algorithms from recent publications, and validate rigorously against both theoretical predictions and practical baselines. My other work follows this same pattern across optimal transport, graph neural networks, and causal inference.

IIEMI

Contents

1 The Business Problem: Why Ambulance Placement Is a Life or Death Decision **5**
1.1 The Scenario . . . 5
1.2 The Hidden Equity Crisis . . . 5
1.3 What This Project Delivers . . . 6
1.4 Quantifying the Impact . . . 6

2 Mathematical Foundations **7**
2.1 Facility Location Problems: The Foundation . . . 8
2.1.1 What Does the p Median Actually Do? . . . 8
2.1.2 The p Median Problem . . . 8
2.1.3 What Does the p Center Actually Do? . . . 9
2.1.4 The p Center Problem . . . 10
2.1.5 Maximal Covering Location Problem (MCLP) . . . 10
2.1.6 Maximum Expected Covering Location Problem (MEXCLP) . . . 11
2.1.7 Summary: Four Models, Four Philosophies . . . 11
2.2 The Equity Constrained Model: Our Novel Contribution . . . 12
2.2.1 The Efficiency Equity Trade off . . . 12
2.2.2 The Key Idea Behind the ECM . . . 12
2.2.3 Our Formulation: Weighted p Median / p Center Hybrid . . . 13
2.3 Measuring Equity: The Gini Coefficient . . . 13
2.4 Network Based Travel Time Computation . . . 14
2.5 Reinforcement Learning for Dynamic Redeployment . . . 15
2.5.1 Why Static Placement Is Not Enough . . . 15
2.5.2 Reinforcement Learning in Plain Terms . . . 15
2.5.3 Markov Decision Process Formulation . . . 15
2.5.4 Proximal Policy Optimization (PPO) . . . 16
2.5.5 Equity Aware Reward Function . . . 17
2.6 The Dual Simulation Architecture . . . 17
2.7 Survival Estimation: Connecting Response Time to Lives Saved . . . 18

3 Datasets **18**
3.1 NYC 911 EMS Incident Dispatch Data . . . 18
3.2 OpenStreetMap Road Network . . . 19
3.3 Supplementary Data . . . 19

4 Code Pipeline: From Raw Data to Results **19**
4.1 Phase 1: Environment Setup (Cells 0a through 0c) . . . 20
4.2 Phase 2: Data Loading and Cleaning (Cells 1a through 1b) . . . 20
4.3 Phase 3: Road Network and OD Matrix (Cells 1c through 1d) . . . 20
4.4 Phase 4: Exploratory Data Analysis (Cells 2a through 2b) . . . 20
4.5 Phase 5: Optimization (Cells 3a through 3f) . . . 21
4.6 Phase 6: Reinforcement Learning (Cells 4a through 4c) . . . 21
4.7 Phase 7: Lives Saved and Summary (Cells 5a through 5b) . . . 21

5 Results **22**
5.1 Static Optimization: Full Model Comparison . . . 22

HLMI

5.2 Pareto Frontier 23
5.3 Reinforcement Learning 24
5.4 Lives Saved Estimation 25

6 Lessons Learned and Iterative Corrections 25
6.1 The Gini Linearization Failure 25
6.2 The Simulation Speed Trap 25
6.3 The Demand Scaling Error 25
6.4 The Partial Road Network 25

7 Limitations and Future Directions 26
7.1 Current Limitations 26
7.2 Future Directions 26

8 Conclusion 26

HILMI

1 The Business Problem: Why Ambulance Placement Is a Life or Death Decision

1.1 The Scenario

Imagine you are the Commissioner of Emergency Medical Services for New York City. You oversee a fleet of approximately 800 ambulances dispatched from stations across five boroughs. Every day, your system receives roughly 4,000 emergency calls. Each call triggers a chain of events: a 911 dispatcher identifies the closest available ambulance, the ambulance travels through city streets to the patient, paramedics provide on scene care, and the patient is transported to a hospital.

Your performance is measured primarily by one metric: **response time**, the elapsed seconds from when a 911 call is received to when the ambulance arrives on scene. The relationship between response time and patient outcomes is not gradual. For out of hospital cardiac arrest (OHCA), which accounts for roughly 1.5% of EMS calls, the survival curve is brutally steep. Research by Holmén et al. (2020) demonstrated that survival probability decreases by approximately 7 to 10 percent for every additional minute of delay. At the eight minute mark, roughly half of untreated patients have irreversible brain damage. At twelve minutes, survival rates approach zero for patients who have not received bystander CPR.

This means that the *geographic placement of your ambulance stations* is, quite literally, a decision about who lives and who dies.

1.2 The Hidden Equity Crisis

The problem extends beyond aggregate speed. It is fundamentally about **fairness**. Existing ambulance station locations in most cities, including New York, reflect decades of historical decisions: a station was built here because land was available, another was placed there because a political leader advocated for it. Over time, population patterns shift, demand changes, neighborhoods gentrify or decline, and certain communities end up systematically underserved.

Research has documented persistent EMS care disparities in the United States linked to race, ethnicity, and socioeconomic status. Crowe et al. (2022) conducted a comprehensive scoping review finding that minority populations consistently receive slower response times and different treatment patterns. A position statement from leading EMS researchers (Dorsett et al., 2024) explicitly established equity as a central goal of EMS quality management, arguing that differences in access to emergency care are "systematic, avoidable, and unjust."

To understand what equity means in this context, consider two neighborhoods. In Neighborhood A (a wealthy area in lower Manhattan), three ambulance stations sit within a two mile radius. When someone calls 911, an ambulance arrives in under six minutes, nearly every time. In Neighborhood B (a lower income area in the southeast Bronx), the nearest station is four miles away through congested streets. Average wait time: over twelve minutes. For a cardiac arrest patient in Neighborhood B, those extra six minutes

are not an inconvenience. They are the difference between walking out of the hospital and never walking again.

The uncomfortable truth is that existing station placement in most cities was not designed to be unfair. It simply was not designed at all. Stations were built when budgets allowed, where land was cheap, or where political influence was strongest. Over decades, these ad hoc decisions calcified into a system where a patient's survival probability depends more on their zip code than on the severity of their condition.

This project asks: *if we could start from scratch, using data instead of politics, where would we put the stations?* And more importantly: *can we do it in a way that explicitly prevents the optimizer from sacrificing some neighborhoods to benefit others?*

The Core Business Questions

This project addresses three questions that any EMS administrator must answer:

1. **Where should we place stations?** Given a fixed budget for p stations, which locations minimize response times across the city?
2. **How do we ensure fairness?** How can we prevent the optimization from concentrating resources in wealthy, high volume neighborhoods while abandoning lower demand areas?
3. **Can we adapt in real time?** Once stations are built, how should we reposition idle ambulances throughout the day as demand patterns shift?

1.3 What This Project Delivers

This project produces a complete, reproducible pipeline that:

1. Analyzes over 18 million historical EMS incidents in New York City to characterize demand patterns and existing response time disparities
2. Computes actual road network travel times using OpenStreetMap data, replacing the unrealistic Euclidean distance assumptions common in academic studies
3. Solves three classical facility location models (p Median, MCLP, MEXCLP) as baselines representing the state of practice
4. Introduces a novel **Equity Constrained Model (ECM)** that explicitly balances efficiency and fairness using a Rawlsian minimax formulation
5. Trains a deep reinforcement learning agent for dynamic ambulance redeployment
6. Estimates lives saved from the proposed improvements using clinical survival models calibrated on published medical data

1.4 Quantifying the Impact

Table 1 previews the core result. The ECM achieves near p Median efficiency while dramatically improving equity.

Table 1: Headline results at $p = 30$ stations. The ECM sacrifices only 19% in mean response time to reduce the worst case by 78%.

Model	Mean RT (s)	Max RT (s)	Gini	Coverage (%)
p Median	150	1800	0.464	98.4
MCLP	210	462	0.364	100.0
ECM (α=0.5)	179	392	0.300	100.0

The p Median leaves at least one zone with a response time of 1800 seconds (the penalty cap for unreachable zones). The ECM eliminates this entirely.

Let us walk through Table 1 one row at a time, because these three rows tell the entire story of this project.

Row 1: p Median. This is the "textbook" solution that most operations research courses would recommend. It places 30 stations in the locations that minimize the average ambulance wait time across the entire city. The average is impressive: 150 seconds, or about two and a half minutes. But look at the "Max RT" column: 1,800 seconds. That is 30 minutes. This means that at least one neighborhood in New York City would have to wait half an hour for an ambulance under this plan. For a heart attack patient, 30 minutes is not a response time. It is a death sentence. The Gini coefficient of 0.464 confirms the inequality: some neighborhoods get two minute responses while others get thirty minute responses.

Row 2: MCLP. This model focuses on coverage: it tries to get as many people as possible within the eight minute threshold. It succeeds (100% coverage), but notice the average response time increased to 210 seconds. The maximum dropped to 462 seconds (about 7.7 minutes), which is much better than p Median. The Gini also improved to 0.364. This is a fairer plan, but it sacrificed efficiency to get there.

Row 3: ECM ($\alpha = 0.5$). This is our contribution. It finds the sweet spot. The average (179 seconds) is only 29 seconds slower than p Median, a modest 19% increase that most patients would not even notice. But the maximum dropped from 1,800 to 392 seconds (6.5 minutes), meaning *every single neighborhood* now gets an ambulance within about six and a half minutes. The Gini fell to 0.300, the most equitable of all three models. Coverage is 100%.

In practical terms: the ECM asks every fast response neighborhood to wait an extra 29 seconds so that the worst served neighborhood goes from a 30 minute wait to a 6.5 minute wait. That is the trade off. Most policymakers, when presented with this choice, would find it compelling.

2 Mathematical Foundations

This section builds the theory from the ground up. Every mathematical concept used in the code is explained here, so that a reader can understand not only *what* the algorithms do, but *why* they work.

2.1 Facility Location Problems: The Foundation

Facility location is a branch of operations research concerned with determining the optimal placement of facilities to serve geographically distributed demand. The field traces to Weber's 1909 work on industrial location and was formalized into modern optimization frameworks by Hakimi (1964) and ReVelle and Swain (1970). In our context, "facilities" are ambulance stations, "demand points" are neighborhoods (ZIP codes) where emergencies occur, and the "cost" of serving a demand point is the travel time from the nearest open station.

Every facility location model shares four components: (1) a set of candidate locations J where stations could be built, (2) a set of demand zones I that need service, (3) a cost matrix t_{ji} giving the travel time from each candidate to each zone, and (4) a budget constraint specifying how many stations p can be opened.

2.1.1 What Does the p Median Actually Do?

Think of the p Median as answering this question: "If I can only build p ambulance stations, where should I put them so that, *on average*, people wait the least amount of time?"

Here is a simplified example. Imagine a city with just three neighborhoods:

- **Neighborhood A:** 1,000 calls per year, located in the city center
- **Neighborhood B:** 500 calls per year, located 3 miles east
- **Neighborhood C:** 50 calls per year, located 10 miles north

If you can build only one station ($p = 1$), where should it go? The p Median says: put it in Neighborhood A (or very close to it). Why? Because A generates the most calls. Placing the station there means 1,000 calls get fast service, 500 calls get moderate service, and 50 calls get slow service. The *weighted average* wait time is minimized.

But notice what happened to Neighborhood C. Those 50 calls, perhaps including heart attacks and strokes, now wait the longest. The p Median does not care about Neighborhood C because it represents only 3% of total demand. Mathematically optimal. Morally questionable.

This is not a hypothetical. In our NYC results, p Median with 30 stations achieved a mean response time of 150 seconds, but at least one ZIP code had a maximum response time of 1,800 seconds (30 minutes). That ZIP code was effectively abandoned.

Now, the formal mathematical statement of this idea:

2.1.2 The p Median Problem

The p Median is the most fundamental efficiency oriented model. It seeks to place exactly p stations to minimize the total demand weighted average response time.

HILMI

Definition 2.1 (p Median Problem)**.** Given demand weights w_i for each zone $i \in I$, travel times t_{ji} from candidate $j \in J$ to zone $i \in I$, and a budget of p stations:

$$\min \quad \frac{1}{W} \sum_{i \in I} \sum_{j \in J} w_i \cdot t_{ji} \cdot y_{ij} \tag{1}$$

$$\text{s.t.} \quad \sum_{j \in J} x_j = p \tag{2}$$

$$\sum_{j \in J} y_{ij} = 1 \quad \forall\, i \in I \tag{3}$$

$$y_{ij} \leq x_j \quad \forall\, i \in I,\, j \in J \tag{4}$$

$$x_j \in \{0, 1\}, \quad y_{ij} \in [0, 1] \tag{5}$$

where $x_j = 1$ if a station is opened at candidate site j, y_{ij} is the fraction of zone i's demand assigned to station j, and $W = \sum_i w_i$ is the total demand.

Why p Median Fails on Equity

The p Median minimizes the *weighted average* response time. Mathematically, this means a zone with 100 daily incidents is 100 times more "important" than a zone with 1 daily incident. The optimizer will place stations near high volume zones and ignore low volume ones, even if those low volume zones have critically ill patients who will die waiting. In our experiments, p Median with $p = 30$ left at least one zone with a maximum response time of 1800 seconds (the penalty cap), while achieving a mean of only 150 seconds. The average looks excellent; the worst case is catastrophic.

This formulation is implemented in the Kaggle notebook in Cell 3a (`solve_p_median` function) and solved using the PuLP library with the CBC open source solver.

2.1.3 What Does the p Center Actually Do?

If the p Median asks "what is best for the average person?", the p Center asks "what is best for the person who is worst off?"

Returning to our three neighborhood example: the p Center would place the station somewhere between A and C, even though this increases the average wait time for the 1,000 callers in Neighborhood A. The logic is that no one should have to wait an unreasonably long time, even if they live in a low demand area.

This philosophy comes from the work of John Rawls, a political philosopher who argued that a just society is one designed to maximize the welfare of its least advantaged members. In our context, the "least advantaged member" is the patient in the ZIP code with the longest ambulance wait time.

The p Center is admirable in principle but impractical in its pure form. Taken to its extreme, it would place stations in the most remote corners of the city to reduce one outlier, while millions of residents in dense neighborhoods see their service degrade. What we need is a balance.

2.1.4 The p Center Problem

The p Center problem takes the opposite approach: instead of minimizing the average, it minimizes the *worst case* response time across all zones.

Definition 2.2 (p Center Problem)**.**

$$\min \quad r_{\max} \tag{6}$$

$$\text{s.t.} \quad r_{\max} \geq r_i \quad \forall\, i \in I \tag{7}$$

$$r_i = \sum_{j \in J} t_{ji} \cdot y_{ij} \quad \forall\, i \in I \tag{8}$$

$$\text{(same assignment and budget constraints as p Median)} \tag{9}$$

The p Center embodies **Rawlsian justice**: it protects the worst off zone, ensuring no community is abandoned. However, it can be extremely inefficient in practice, placing stations in remote areas to reduce one outlier while degrading service for thousands of people in dense neighborhoods.

2.1.5 Maximal Covering Location Problem (MCLP)

The MCLP, introduced by Church and ReVelle (1974), takes a threshold based approach. Instead of optimizing a continuous metric, it maximizes the population *covered* within a specified response time threshold (for example, 480 seconds, or eight minutes).

$$\max \quad \sum_{i \in I} w_i \cdot z_i \tag{10}$$

$$\text{s.t.} \quad z_i \leq \sum_{j \in J} a_{ij} \cdot x_j \quad \forall\, i \in I \tag{11}$$

$$\sum_{j \in J} x_j = p \tag{12}$$

$$x_j, z_i \in \{0, 1\} \tag{13}$$

where $z_i = 1$ if zone i is covered, and $a_{ij} = 1$ if $t_{ji} \leq T_{\text{threshold}}$. This formulation is implemented in Cell 3a (`solve_mclp`).

In plain terms, the MCLP asks: "With p stations, how many people can I get an ambulance to within eight minutes?" It does not care about *how fast* the ambulance arrives, only whether it arrives within the threshold. A zone served in two minutes and a zone served in seven minutes and fifty nine seconds are equally "covered." A zone served in eight minutes and one second is "uncovered," even though it is only one second over the line.

This binary thinking is both the strength and weakness of the MCLP. It is easy for policymakers to understand ("92% of the population is covered within eight minutes") but it ignores the severity of failures. A zone that misses the threshold by one second is treated the same as a zone that misses it by twenty minutes.

HILMI

2.1.6 Maximum Expected Covering Location Problem (MEXCLP)

The MEXCLP (Daskin, 1983) extends the MCLP by incorporating the reality that ambulances are not always available when a call arrives. It introduces the **busy fraction** q: the probability that any given ambulance is occupied. If only one ambulance covers a zone and it is busy (probability q), the zone is effectively uncovered. Having multiple ambulances covering the same zone provides backup.

The expected coverage of zone i served by k ambulances is:

$$E[\text{coverage}_i] = \sum_{l=1}^{k} (1-q) \cdot q^{l-1} \tag{14}$$

The busy fraction q is estimated using an M/M/c queueing approximation:

$$q = \frac{\lambda}{c \cdot \mu} \tag{15}$$

where λ is the total call arrival rate, c is the number of ambulances, and μ is the service rate (inverse of mean service time per call).

Why Queueing Theory Matters Here

Classical facility location models assume that when a call arrives, the nearest ambulance is always available. In NYC, where ambulances handle over 4,000 calls per day, this assumption is wildly unrealistic. During peak hours, the busy fraction can exceed 0.5, meaning that half the time, the nearest ambulance is already occupied. The MEXCLP accounts for this by valuing zones with multiple covering ambulances more highly than zones with only one.

2.1.7 Summary: Four Models, Four Philosophies

Before introducing our novel contribution, let us pause and summarize what each model prioritizes. Understanding these philosophies is essential because our ECM is designed to combine the best aspects of each.

Table 2: Comparison of facility location model philosophies.

Model	What It Optimizes	Strength	Weakness
p Median	Average response time	Best overall efficiency	Ignores worst off zones
p Center	Worst case response time	Protects every zone	Sacrifices average quality
MCLP	Population within threshold	Easy to communicate	Binary, ignores severity
MEXCLP	Expected coverage with busy ambulances	Realistic about availability	Still threshold based

None of these models alone solves the problem. The p Median abandons outlier zones. The p Center over protects them at everyone else's expense. The MCLP and MEXCLP use a crude binary threshold. What we need is a model that says: "minimize the average wait time, *but never let any zone wait longer than a specified maximum*." That is exactly what the ECM does.

2.2 The Equity Constrained Model: Our Novel Contribution

2.2.1 The Efficiency Equity Trade off

The fundamental tension in ambulance placement is this: **efficiency and equity are competing objectives**. Placing all stations in Manhattan (the densest borough) would minimize the average response time because more calls occur there. But it would abandon the Bronx, Queens, Brooklyn, and Staten Island.

This trade off is well documented in the operations research literature. Bertsimas et al. (2011) formalized "the price of fairness," showing that any departure from pure efficiency optimization incurs a cost, and characterizing the Pareto frontier between efficiency and equity. Enayati et al. (2023) provided the first systematic analysis of equity in ambulance location using the Gini coefficient and Rawlsian justice principles. Jagtenberg and Mason (2020) proposed time sharing between ambulance configurations to improve fairness. Aringhieri et al. (2024) developed an integrated location simulation framework using isochrone coverage and Gini indices. Most recently, Saadatmand et al. (2025) built a data driven decision support system integrating horizontal and vertical equity perspectives.

2.2.2 The Key Idea Behind the ECM

Imagine you are tuning a radio dial. On the far left ($\alpha = 0$), the radio plays "protect the worst off zone at all costs" (pure p Center). On the far right ($\alpha = 1$), it plays "minimize the average, ignore the outliers" (pure p Median). As you turn the dial, you smoothly transition between these two extremes.

At $\alpha = 0.5$ (the middle of the dial), the model says: "I care equally about the average and the worst case. I will accept a slightly longer average wait time if it means dramatically reducing the longest wait." In our results, this setting increased the average from 150 seconds to 179 seconds (a modest 19% increase) while reducing the maximum from 1,800 seconds to 392 seconds (a dramatic 78% reduction). In practical terms: every neighborhood now gets an ambulance within about six and a half minutes, at the cost of the very fastest neighborhoods waiting an extra half minute.

The mathematical formulation below makes this dial precise. The parameter α is the dial position, and the optimizer finds the best station placement for whatever position you choose. By sweeping α from 0 to 1, we generate the full menu of options, called the "Pareto frontier," that a policymaker can choose from.

2.2.3 Our Formulation: Weighted p Median / p Center Hybrid

Our ECM combines the p Median objective (efficiency) and the p Center objective (equity) into a single tunable formulation:

Definition 2.3 (Equity Constrained Model (ECM)).

$$\min \quad \alpha \cdot \underbrace{\frac{1}{W}\sum_{i\in I} w_i \cdot r_i}_{\text{Mean RT (efficiency)}} + (1-\alpha) \cdot \underbrace{r_{\max}}_{\text{Max RT (equity)}} \tag{16}$$

$$\text{s.t.} \quad \sum_{j\in J} x_j = p \tag{17}$$

$$\sum_{j\in J} y_{ij} = 1 \quad \forall\, i \in I \tag{18}$$

$$y_{ij} \le x_j \quad \forall\, i, j \tag{19}$$

$$r_i = \sum_{j\in J} t_{ji} \cdot y_{ij} \quad \forall\, i \in I \tag{20}$$

$$r_{\max} \ge r_i \quad \forall\, i \in I \tag{21}$$

$$x_j \in \{0,1\},\ y_{ij} \in [0,1],\ r_i \ge 0 \tag{22}$$

The parameter $\alpha \in [0,1]$ is a policy dial:

- $\alpha = 1.0$: Pure p Median. Minimize average, ignore worst case.
- $\alpha = 0.0$: Pure p Center. Protect worst off, ignore average.
- $\alpha = 0.5$: Balanced trade off. Our recommended default for policy use.

By sweeping α from 1.0 to 0.0, we generate the **Pareto frontier**: the set of all solutions where no improvement in one objective is possible without worsening the other. This frontier is the central analytical deliverable of the project.

This formulation is implemented in Cell 3c (`solve_ecm_v2`) of the Kaggle notebook.

Why We Abandoned the Gini Linearization

An earlier version attempted to constrain the Gini coefficient directly using a Mean Absolute Deviation (MAD) approximation: Gini $\approx$ MAD$/(2 \cdot \bar{r}) \le \varepsilon$. This produced catastrophic results: mean response times of 606 seconds with Gini values of 0.53, worse than p Median on *both* dimensions. The solver satisfied the Gini constraint by making all zones *equally slow* rather than reducing disparity. The weighted p Median / p Center hybrid avoids this entirely because the minimax term $r_{\max}$ directly penalizes the worst case without distorting the mean. Section 6 provides the full account of this correction.

2.3 Measuring Equity: The Gini Coefficient

Most people have heard of the Gini coefficient in the context of income inequality. A country where everyone earns the same salary has a Gini of 0 (perfect equality). A

HLM

country where one person earns everything and everyone else earns nothing has a Gini of 1 (maximum inequality).

We apply the same idea to ambulance response times. If every ZIP code in NYC experienced the same response time, the Gini would be 0. If one ZIP code experienced all the delay while others had instant service, the Gini would approach 1. In our data, the current Gini across NYC ZIP codes is 0.078, which sounds low but masks significant within borough disparities.

When we run the p Median optimizer, the Gini jumps to 0.46 because the model concentrates stations in high demand areas and abandons low demand ones. The ECM at $\alpha = 0.5$ brings this down to 0.30, a 35% reduction that means the gap between the best served and worst served neighborhoods is substantially narrowed.

The Gini coefficient, originally developed by Corrado Gini in 1912 to measure income inequality, is adapted here to quantify the inequality of response times across zones. A Gini of 0 means all zones experience identical response times (perfect equality). A Gini of 1 means all response time burden falls on a single zone (maximum inequality).

For demand weighted observations sorted in ascending order of response time:

$$G = 1 - \frac{2 \sum_{k=1}^{n} C_k \cdot w_{\sigma(k)}}{W \cdot \sum_{k=1}^{n} w_{\sigma(k)} \cdot r_{\sigma(k)}} \tag{23}$$

where σ is the permutation sorting response times, C_k is the cumulative weighted response time, and W is total demand.

This metric is computed in the notebook via the `gini()` utility function (Cell 3a) and is used both as an evaluation metric and implicitly within the RL reward function.

2.4 Network Based Travel Time Computation

A critical distinction of this project from many academic studies is the use of **actual road network travel times** rather than Euclidean (straight line) distance. In a dense urban environment like New York City, the straight line distance can differ from driving distance by a factor of 1.5 to 3.0 due to one way streets, bridges, tunnels, and the irregular grid of outer boroughs.

We use **OSMnx** (Boeing, 2025), a Python library that downloads, models, and analyzes street networks from OpenStreetMap. The network is a directed graph $G = (V, E)$ where nodes V are intersections and edges E are road segments with attributes: length (meters), speed (km/h, from OSM tags with imputation), and computed travel time (seconds).

Dijkstra's algorithm computes shortest path travel times between all pairs of demand zone centroids and candidate station locations. The time complexity is $O((|V| + |E|) \log |V|)$ per source node. For the full OD matrix with 237 zones, this requires 237 Dijkstra runs, each traversing a graph with 55,268 nodes and 139,160 edges.

This computation is performed in Cell 1d of the Kaggle notebook.

To understand why this matters, consider two points in NYC that are 2 kilometers apart in a straight line: one on the Manhattan side of the East River, one on the Brooklyn side. A Euclidean distance model would estimate a 3 minute ambulance ride. In reality,

the ambulance must cross a bridge, potentially the Manhattan Bridge or the Brooklyn Bridge, adding detours, traffic, and toll plaza slowdowns. The actual drive time could be 12 to 15 minutes. Using Euclidean distance would lead the optimizer to believe these two points are easily connected, potentially skipping a station placement on the Brooklyn side that is desperately needed.

This is not an edge case. NYC's geography is defined by rivers, bridges, tunnels, one way streets, and the irregular grid of outer boroughs. Studies that use Euclidean distance in NYC are not just imprecise; they can produce station placements that are fundamentally wrong.

2.5 Reinforcement Learning for Dynamic Redeployment

2.5.1 Why Static Placement Is Not Enough

The optimization models in Section 2.2 determine *where to build stations*, a strategic decision made once. But within any given day, the positioning of individual ambulances changes constantly. An ambulance dispatched to an emergency in the Bronx may transport the patient to a hospital in Manhattan, leaving the Bronx temporarily underserved. Dynamic redeployment, the practice of repositioning idle ambulances to anticipate future demand, is an operational decision made every few minutes.

2.5.2 Reinforcement Learning in Plain Terms

Reinforcement learning (RL) is a type of artificial intelligence where a computer agent learns by trial and error, similar to how a child learns to ride a bicycle. The child tries different actions (lean left, lean right, pedal faster), observes the result (stay upright or fall), and gradually learns a policy (a set of rules) that keeps them balanced.

In our case, the "child" is an algorithm managing ambulance positions. Every five minutes, it observes the current state of the city (where are ambulances now? which ones are busy? what time of day is it? where are calls coming from?) and decides whether to reposition any idle ambulances. After making a decision, it observes the outcome (how long did patients wait?) and receives a score (the reward). Over millions of practice rounds in a simulated city, the algorithm learns which repositioning strategies lead to the shortest, fairest response times.

The formal framework for this is called a **Markov Decision Process** (MDP), which is simply a mathematical way of describing any situation where an agent makes sequential decisions under uncertainty:

2.5.3 Markov Decision Process Formulation

We model dynamic redeployment as a **Markov Decision Process (MDP)**, the standard mathematical framework for sequential decision making under uncertainty. An MDP is defined by four components:

Definition 2.4 (Markov Decision Process)**.** An MDP consists of:

- **State** s_t: A complete description of the system at time t. In our formulation, this includes ambulance counts per zone, busy status per zone, current demand rate per zone, pending call counts, and time of day features (sine/cosine encoded hour, day of week, weekend flag, night flag, rush hour flag).
- **Action** a_t: The redeployment decision. We use a hierarchical design: for each of the five NYC boroughs, the agent selects a redistribution level (0 to 9) controlling how to spread idle ambulances between even distribution and demand proportional concentration.
- **Transition** $P(s_{t+1}|s_t, a_t)$: The probabilistic dynamics. New calls arrive via Poisson processes; service completions follow geometric distributions; ambulance positions update based on the chosen action.
- **Reward** $R(s_t, a_t)$: The signal that guides learning. We use a constrained formulation (see below).

2.5.4 Proximal Policy Optimization (PPO)

We solve the MDP using **Proximal Policy Optimization** (Schulman et al., 2017), a deep reinforcement learning algorithm that has become the de facto standard for continuous control problems. PPO maintains two neural networks: a *policy network* $\pi_\theta(a|s)$ that maps states to actions, and a *value network* $V_\phi(s)$ that estimates expected future reward from each state.

PPO's key innovation is a clipped objective function that prevents destructively large policy updates:

$$L^{\text{CLIP}}(\theta) = \mathbb{E}\left[\min\left(\frac{\pi_\theta(a|s)}{\pi_{\theta_{\text{old}}}(a|s)}\hat{A},\ \text{clip}\left(\frac{\pi_\theta(a|s)}{\pi_{\theta_{\text{old}}}(a|s)}, 1-\epsilon, 1+\epsilon\right)\hat{A}\right)\right] \tag{24}$$

where $\hat{A}$ is the advantage estimate (how much better the chosen action was than expected) and $\epsilon = 0.2$ is the clipping range.

In simpler terms, PPO works like this: the agent tries a policy (a set of rules for repositioning ambulances), evaluates how well it worked by comparing outcomes to expectations, and then adjusts the rules slightly. The "proximal" part means it only makes small adjustments at each step, preventing the agent from making a drastic change that works well in one situation but catastrophically fails in others. Think of it as a cautious learner: it improves steadily without overreacting to any single experience.

The "clipping" mechanism is the mathematical guardrail that enforces this caution. If the new policy would deviate too much from the old one (more than 20% in probability), the update is clipped, keeping the learning process stable.

This approach follows recent work by Liu and Zeng (2024) who applied PPO to ambulance dispatch using OSM travel times (presented at ICLR 2024), and Sivagnanam et al. (2024) who introduced hierarchical multi agent RL for emergency responder stationing (ICML 2024). Earlier foundational work includes Liu et al. (2020) who proposed multi agent Q learning for ambulance dispatch (ACM SIGSPATIAL 2020) and Allen et al. (2021) who developed an OpenAI Gym compatible simulation environment for ambulance location RL.

2.5.5 Equity Aware Reward Function

The reward function determines what the RL agent optimizes. A naive reward of $R = -\bar{r}$ (negative mean response time) would produce an agent that behaves like p Median: efficient on average, inequitable in practice. We use a **constrained reward** that penalizes equity violations:

$$R = -\bar{r} - \lambda_G \cdot \max(0,\, G - \varepsilon_G) - \lambda_W \cdot \max(0,\, (r_{90} - T_{\max})/T_{\max}) \tag{25}$$

where $\bar{r}$ is the mean response time, G is the Gini coefficient, r_{90} is the 90th percentile response time, $\lambda_G = 100$ and $\lambda_W = 50$ are penalty multipliers, $\varepsilon_G = 0.15$ is the Gini threshold, and $T_{\max} = 720$ seconds is the worst case threshold.

To understand this reward function, imagine you are grading the agent's performance. The base grade is determined by how fast ambulances arrived on average (the $\bar{r}$ term). If the agent kept response times fair across neighborhoods (Gini below the threshold), it gets no additional penalty. But if some neighborhoods were unfairly slow (Gini exceeds 0.15), the agent receives a large grade deduction ($\lambda_G = 100$ times the violation amount). Similarly, if the 90th percentile response time exceeds 12 minutes, another penalty kicks in.

This design creates a clear priority ordering: **first be fair, then be fast**. The agent quickly learns that violating the equity constraints is extremely costly, so it satisfies them first. Once equity is maintained, it focuses on reducing the average response time within those bounds.

The penalty terms activate *only when constraints are violated*. When the agent maintains equity, the reward simplifies to $R = -\bar{r}$ (pure efficiency). This design drives the agent to first satisfy equity thresholds, then optimize efficiency within those bounds.

The RL environment and training are implemented in Cells 4a and 4b of the Kaggle notebook.

2.6 The Dual Simulation Architecture

Why Two Simulation Engines?

Training a PPO agent requires millions of environment interactions. A full fidelity discrete event simulation (SimPy) processes events sequentially, making it far too slow for RL training. Our solution: a **vectorized NumPy environment** for training (pure array operations, approximately 1000 times faster) and a SimPy based discrete event simulation reserved for final policy evaluation only. This architectural decision was adopted after recognizing that training PPO inside SimPy would require weeks of computation on a single GPU. See Section 6 for the full reasoning.

HILMI

2.7 Survival Estimation: Connecting Response Time to Lives Saved

To translate response time improvements into a metric that resonates with policymakers and the public, we apply the clinical survival model from Holmén et al. (2020):

$$P(\text{survival}) = P_0 \cdot (1 - \delta)^{r/60} \tag{26}$$

where $P_0 = 0.12$ is the baseline OHCA survival rate, $\delta = 0.07$ is the per minute decay factor, and r is the response time in seconds. The annual lives saved from a response time improvement are:

$$\Delta L = N \cdot f_{\text{OHCA}} \cdot \left[P_0(1 - \delta)^{r_{\text{new}}/60} - P_0(1 - \delta)^{r_{\text{old}}/60}\right] \tag{27}$$

where $N = 1{,}500{,}000$ is the annual call volume and $f_{\text{OHCA}} = 0.015$ is the cardiac arrest fraction.

To make this concrete: if our optimization reduces the mean response time from 8.7 minutes (the current NYC average) to 7.4 minutes (a 15% improvement), the survival rate for cardiac arrest patients increases from about 6.3% to about 6.9%. That 0.6 percentage point improvement, applied to approximately 22,500 annual cardiac arrest calls in NYC, translates to roughly 142 additional people who survive. These are 142 parents who come home to their families, 142 colleagues who return to work, 142 neighbors who are still alive because an ambulance station was in the right place.

This is why the title of this document is not "An Optimization Framework for Emergency Vehicle Deployment." It is "When Minutes Mean Lives." Because that is what the mathematics ultimately describes: the relationship between an algorithm's output and whether someone's loved one comes home.

HLMI

3 Datasets

3.1 NYC 911 EMS Incident Dispatch Data

The primary dataset is the EMS Incident Dispatch Data published by the Fire Department of New York (FDNY) through the NYC Open Data portal and mirrored on Kaggle. This dataset is generated by FDNY's Computer Aided Dispatch (CAD) system and records every emergency medical incident from the moment a 911 call is received through the closure of the incident.

- **Source:** NYC Open Data (https://data.cityofnewyork.us/Public-Safety/EMS-Incident-Di 76xm-jjuj) and Kaggle (https://www.kaggle.com/datasets/new-york-city/ny-ems-incident
- **Records:** Over 20 million incidents
- **Temporal coverage:** 2005 through 2020
- **Spatial granularity:** ZIP code level (aggregated for HIPAA compliance)

The key fields used in this project are listed in Table 3.

Table 3: Key fields from the EMS Incident Dispatch Data.

Field	Description
INCIDENT_DATETIME	Timestamp when the incident was created in the CAD system
INCIDENT_RESPONSE_SECONDS_QY	Total response time in seconds from call received to ambulance on scene
INCIDENT_TRAVEL_TM_SECONDS_QY	Travel time in seconds from dispatch to scene arrival
INITIAL_SEVERITY_LEVEL_CODE	Severity classification at dispatch (1 to 8 scale)
ZIPCODE	ZIP code of the incident location (HIPAA aggregated)
BOROUGH	Manhattan, Bronx, Brooklyn, Queens, or Staten Island
HELD_INDICATOR	Whether the call was held due to no available units (Y/N)

HIPAA and Spatial Granularity

Due to the Health Insurance Portability and Accountability Act, the dataset does not include exact incident addresses. Locations are aggregated to the ZIP code level. In the optimization models, each ZIP code is represented by its geographic centroid. A planned sensitivity analysis would perturb centroids by up to 1 km in random directions to assess robustness of station placement to this aggregation.

3.2 OpenStreetMap Road Network

The road network is obtained from OpenStreetMap using OSMnx (Boeing, 2025). The current run produced a graph with 55,268 nodes and 139,160 edges, covering the core urban area. Edge speeds are inferred from OSM `maxspeed` tags with imputation for missing values.

3.3 Supplementary Data

NYC ZIP Code boundary polygons were obtained from the NYC Open Data geospatial API for computing centroids and spatial adjacency.

4 Code Pipeline: From Raw Data to Results

This section walks through the Kaggle notebook (`ambulance-optimization.ipynb`) cell by cell. The reader should follow along with the notebook open.

Notebook: `ambulance-optimization.ipynb` on Kaggle
Dataset: Add "NY EMS Incident Dispatch Data" from Kaggle datasets
Runtime: GPU T4 with internet enabled
Total runtime: Approximately 2 to 3 hours

4.1 Phase 1: Environment Setup (Cells 0a through 0c)

Cell 0a installs additional packages: `osmnx`, `h3`, `simpy`, `pulp`, `stable-baselines3`, `sodapy`, and `folium`. Cell 0b imports all libraries and verifies GPU availability (Tesla T4). Cell 0c locates the EMS CSV by scanning all files in `/kaggle/input/` and selecting the largest (5.21 GB).

4.2 Phase 2: Data Loading and Cleaning (Cells 1a through 1b)

Cell 1a loads only eight required columns (to avoid kernel memory crashes on Kaggle's 13 GB RAM). All columns are read as strings to minimize memory overhead.

Cell 1b performs cleaning: parsing dates and numerics, filtering to valid response times (0 to 3600 seconds), validating NYC ZIP codes (10001 to 11697), standardizing borough names, and engineering time features (hour, day of week, weekend indicator, four hour time window). The pipeline retains 87.2% of records: 18,102,593 out of 20,755,333.

4.3 Phase 3: Road Network and OD Matrix (Cells 1c through 1d)

Cell 1c downloads the NYC road network from OpenStreetMap via OSMnx and adds speed and travel time attributes. Cell 1d geocodes all 237 ZIP code centroids, maps them to the nearest graph nodes, and computes the full 237×237 OD matrix using Dijkstra's algorithm. Mean travel time in the OD matrix is 976 seconds.

4.4 Phase 4: Exploratory Data Analysis (Cells 2a through 2b)

The EDA produces four plots saved as `eda_overview.png` and a borough level summary table (Table 4).

Table 4: Borough level response time statistics.

Borough	Incidents	Mean (min)	Median (min)	P90 (min)	Under 8 min
Bronx	4,200,772	9.08	7.40	15.47	56.3%
Manhattan	4,405,685	8.99	7.00	16.20	59.7%
Brooklyn	5,293,148	8.52	7.12	14.43	59.4%
Queens	3,449,028	8.16	7.12	13.37	59.9%
Staten Island	753,960	7.45	6.57	12.12	66.5%

HILMI

What does this table tell us? It reveals the equity problem hiding in plain sight.

Look at the "Under 8 min" column. In Staten Island, 66.5% of calls receive an ambulance within eight minutes. In the Bronx, only 56.3% do. That 10 percentage point gap means that roughly 420,000 Bronx calls per year wait longer than eight minutes compared to what Staten Island residents experience. The Bronx has the highest mean response time (9.08 minutes), the highest P90 (15.47 minutes), and the lowest coverage rate.

Now look at the "Incidents" column. Brooklyn generates the most calls (5.3 million), but the Bronx, despite having fewer calls, has worse performance. This suggests the Bronx is not just high demand; it is *underserved relative to its demand*. Stations are not positioned proportionally to where people need help.

The "P90" column is particularly telling. P90 means "90% of calls are answered within this time." For Manhattan, P90 is 16.2 minutes, which means 10% of Manhattan calls wait longer than 16 minutes. For a borough with world class hospitals on every other block, this is surprisingly poor. It suggests that even in well resourced areas, the tail of the response time distribution contains serious failures.

These patterns are exactly what our optimization models are designed to address. The p Median will make the averages better but may worsen the tails. The ECM will explicitly prevent any borough from experiencing response times above a specified maximum.

Insert Figure: `eda_overview.png`

HIIMI

4.5 Phase 5: Optimization (Cells 3a through 3f)

Cell 3a defines the `gini()` utility and `zone_rt_from_selection()` helper. Cells 3b defines p Median, MCLP, and their solvers. Cell 3c defines the ECM v2 (`solve_ecm_v2`). Cell 3d runs all models for $p \in \{30, 50, 70\}$ with the ECM sweeping $\alpha \in \{1.0, 0.8, 0.6, 0.5, 0.4, 0.2, 0.0\}$. Cell 3e plots the Pareto frontier. Cell 3f plots model comparison bar charts.

Output files: `model_comparison.csv`, `pareto_frontier.csv`, `pareto_frontier.png`, `model_comparison.png`.

4.6 Phase 6: Reinforcement Learning (Cells 4a through 4c)

Cell 4a defines the `VecEMS` class, a vectorized NumPy based environment. Cell 4b trains the PPO agent for 1,000,000 steps across 8 parallel environments (approximately 50 minutes on T4). Cell 4c evaluates the trained policy against a random baseline over 20 episodes.

Output file: `ppo_ambulance.zip`.

4.7 Phase 7: Lives Saved and Summary (Cells 5a through 5b)

Cell 5a computes lives saved estimates. Cell 5b prints the final project summary.

5 Results

5.1 Static Optimization: Full Model Comparison

Table 5: Complete optimization results across all models and station counts.

Model	p	Mean RT (s)	Gini	P90 RT (s)	Max RT (s)	Coverage
p Median	30	150	0.464	369	1800	98.4%
MCLP	30	210	0.364	392	462	100.0%
ECM (α=0.5)	30	179	0.300	321	392	100.0%
p Median	50	98	0.582	318	1800	99.4%
MCLP	50	171	0.486	388	475	100.0%
ECM (α=0.5)	50	141	0.400	285	321	100.0%
p Median	70	64	0.722	284	1800	99.8%
MCLP	70	134	0.581	352	475	100.0%
ECM (α=0.5)	70	103	0.500	224	238	100.0%

LILIMI

The key finding: **ECM achieves near p Median efficiency while dramatically improving equity.** At $p = 30$, ECM's mean response time (179s) is only 19% higher than p Median (150s), but it reduces the maximum response time from 1800s to 392s (a 78% reduction) and lowers the Gini from 0.464 to 0.300 (a 35% reduction).

The 1800 Second Penalty

The p Median maximum of 1800 seconds is not a real travel time. It is the penalty cap assigned to zones where no selected station can reach them via the road network. This means p Median *completely abandons* at least one ZIP code. The ECM eliminates this: every zone receives service within 392 seconds (6.5 minutes) at $p = 30$.

Let us read the patterns in Table 5 that matter most.

Pattern 1: More stations improve everything, but with diminishing returns. As we increase from 30 to 50 to 70 stations, every model improves. The p Median mean drops from 150 to 98 to 64 seconds. But the improvement from 30 to 50 stations (52 seconds) is larger than from 50 to 70 (34 seconds). This diminishing return is important for budget decisions: the first 30 stations do most of the heavy lifting.

Pattern 2: p Median's Gini *worsens* with more stations. This is counterintuitive but critical. At $p = 30$, p Median's Gini is 0.464. At $p = 70$, it is 0.722. More stations made the inequality *worse*. Why? Because each additional station goes to a high demand area (where it reduces the average the most), making the gap between well served and poorly served zones even wider. The extra stations help the neighborhoods that already had good service.

Pattern 3: The ECM maintains consistent equity. The ECM's Gini stays in a narrow range (0.300 to 0.500) regardless of how many stations are added. More importantly, the maximum response time drops steadily: 392 seconds at $p = 30$, 321 at $p = 50$,

238 at $p = 70$. Every additional station benefits both the average *and* the worst case. This is the hallmark of an equity aware design.

Pattern 4: p Median always leaves zones behind. At every p value, p Median's maximum is 1,800 seconds. Whether you build 30 or 70 stations, at least one zone remains unreachable. The ECM and MCLP never do this.

The bottom line for a decision maker: If you care only about the citywide average, use p Median. If you also care about the worst served neighborhood (and you should), use ECM at $\alpha = 0.5$. The cost is modest (19 to 60% higher average, depending on p), but the benefit is dramatic (no zone left behind).

Insert Figure: `model_comparison.png`

5.2 Pareto Frontier

The Pareto frontier (saved as `pareto_frontier.png`) traces the full range of achievable solutions. As α decreases from 1.0 to 0.0, mean response time increases but maximum response time drops sharply. The frontier is convex: small sacrifices in average performance yield large reductions in worst case time, with diminishing returns beyond $\alpha = 0.3$.

For readers unfamiliar with the term: a **Pareto frontier** is a curve that shows all possible "best" trade offs between two competing goals. In our case, the two goals are:

- **Efficiency** (horizontal axis): How fast is the average response? (lower is better)
- **Equity** (vertical axis): How fast is the worst case response? (lower is better)

Every point on the frontier represents a station configuration where you cannot improve one goal without worsening the other. Points below and to the left are "better" (faster average AND faster worst case), but no feasible solution exists there with the given number of stations.

How a policymaker would use this chart:

1. Start at the far right of the curve ($\alpha = 1.0$, pure p Median). This is the fastest average but worst equity.
2. Move left along the curve. Each step slightly increases the average but dramatically decreases the worst case.
3. At some point, the curve flattens. Further equity improvements require large sacrifices in efficiency.
4. The "elbow" of the curve, typically around $\alpha = 0.4$ to 0.6, represents the best balance. This is where a small efficiency sacrifice buys a large equity gain.

The policymaker does not need to understand the mathematics. They look at the chart, decide how much average performance they are willing to sacrifice for equity, and point to the corresponding spot on the curve. The model then tells them exactly where to place the stations.

Insert Figure: `pareto_frontier.png`

5.3 Reinforcement Learning

The PPO agent achieved a mean response time of 21 seconds compared to 25 seconds for random redeployment, a **15.3% improvement**. Training logs show convergence with explained variance of 0.957, indicating the value function learned an effective state representation.

Comparing RL to Static Optimization

The RL response times (21 to 25 seconds) are much lower than optimization results (100 to 180 seconds) because the RL environment uses a simplified queueing approximation with different demand scaling. The RL result demonstrates the *relative* benefit of learned redeployment over random redeployment, not an absolute prediction. A SimPy based evaluation would be needed for direct comparison.

To understand the RL result in context: the 15.3% improvement over random redeployment means that intelligent, learned repositioning of idle ambulances provides a meaningful additional benefit *on top of* good station placement. Think of it this way:

HLMI

- **Static optimization** (the ECM) answers: "Where should we *build* stations?" This is a one time decision.
- **Reinforcement learning** answers: "Given the stations we built, where should idle ambulances *wait* right now?" This is a continuous, minute by minute decision.

Even with perfectly placed stations, ambulances move throughout the day. An ambulance dispatched from the Bronx may transport a patient to a Manhattan hospital, leaving the Bronx temporarily underserved. The RL agent learns to anticipate these gaps and proactively reposition idle ambulances to maintain coverage.

The 15.3% improvement is consistent with published results in the field. Liu and Zeng (2024) reported similar magnitudes using PPO on Norwegian EMS data, and Sivagnanam et al. (2024) demonstrated 10 to 25% improvements with hierarchical multi agent RL. These consistent findings across different cities and datasets suggest that dynamic redeployment is a robust, generalizable benefit.

The key takeaway: optimal station placement gets you 80% of the way there. Intelligent redeployment gets you the remaining 20%. Both matter.

5.4 Lives Saved Estimation

Table 6: Estimated additional OHCA survivors per year from response time improvements.

Improvement	New Mean RT (min)	Lives Saved / Year
10%	7.8	93
15%	7.4	142
20%	6.9	193
25%	6.5	245
30%	6.1	299

Even a modest 10% improvement saves an estimated 93 lives per year from cardiac arrest alone.

These numbers deserve careful interpretation. They represent *additional* survivors of out of hospital cardiac arrest (OHCA) only. OHCA accounts for roughly 1.5% of total EMS calls. The full impact of faster response times extends to stroke patients (where every minute of delay destroys 1.9 million neurons), severe trauma patients (where the "golden hour" determines surgical outcomes), and overdose patients (where timely naloxone administration reverses opioid effects).

If we conservatively assume that response time improvements benefit all time sensitive emergencies, not just cardiac arrest, the true impact could be three to five times higher than the OHCA estimates in Table 6.

To put 142 lives per year (the 15% improvement scenario) in perspective: this is roughly equivalent to preventing all traffic fatalities in a mid size American city. It would be achieved not by building new hospitals, hiring more paramedics, or purchasing new equipment, but simply by *repositioning existing resources more intelligently.* The ambulances, the paramedics, and the equipment already exist. The only thing that changes is where they wait between calls.

This is the fundamental promise of operations research applied to public health: better outcomes from the same resources, achieved through better decisions.

6 Lessons Learned and Iterative Corrections

6.1 The Gini Linearization Failure

The first ECM formulation constrained the Gini coefficient directly via a Mean Absolute Deviation approximation: Gini $\approx$ MAD$/(2\bar{r}) \leq \varepsilon$. With $\varepsilon = 0.15$ and $T_{\max} = 720$s, the solver produced mean RT of 606s and Gini of 0.53, *worse than p Median on both dimensions.* The constraint was satisfied by making all zones equally slow. The fix: the weighted p Median / p Center hybrid, which is always feasible and monotone.

6.2 The Simulation Speed Trap

The initial plan trained PPO inside SimPy. After estimation (millions of steps at seconds per step), this would take weeks. The fix: dual simulation architecture with vectorized NumPy for training and SimPy for evaluation only, following Allen et al. (2021).

6.3 The Demand Scaling Error

The first RL run produced zero rewards throughout 1M steps. The demand rate was near zero: dividing total demand (18M) into per zone per interval rates yielded 0.001 calls/interval. The environment never generated calls. The fix: computing actual calls per five minute interval distributed proportionally.

6.4 The Partial Road Network

Full NYC should have 250K+ nodes; we obtained 55K. A borough by borough fallback was implemented but Kaggle timeouts limited coverage. Acknowledged as a limitation.

HLMI

7 Limitations and Future Directions

7.1 Current Limitations

1. **ZIP code granularity:** HIPAA aggregation means exact locations are unknown. Centroid assumption introduces up to 1 to 2 km spatial error.

2. **Static demand:** Optimization models use time averaged demand. Reality varies by hour, season, and events.

3. **Partial road network:** 55K nodes covers the core urban area. Full coverage would refine outer borough travel times.

4. **Simplified RL environment:** Queueing approximation, not full DES. RL metrics are not directly comparable to static optimization.

5. **No real time traffic:** Free flow speeds only. Congestion, construction, and weather are not modeled.

6. **Dataset temporal coverage:** 2005 to 2020. Post pandemic patterns are not captured.

7.2 Future Directions

1. **Spatio temporal demand forecasting:** An ST GCN could predict demand at ZIP level across four hour windows.

2. **Multi objective RL:** A MORL framework could learn the full Pareto set of redeployment policies.
3. **Hospital capacity integration:** Extend to include ED wait times and patient transport routing.
4. **Transferability:** Apply to cities with different geometries (Los Angeles, Pittsburgh, Jakarta).
5. **Drone ambulance integration:** Joint optimization of ground stations and drone launch sites.
6. **Climate resilience:** Robust optimization for demand spikes during extreme weather.

8 Conclusion

This project demonstrates that mathematical optimization and machine learning can produce ambulance configurations that are simultaneously efficient and equitable. The ECM, formulated as a weighted p Median / p Center hybrid, achieves near optimal average response times while reducing worst case times by up to 78%.

The Pareto frontier provides a transparent tool for policymakers: rather than a single "optimal" solution, it presents the full range of trade offs, allowing decision makers to choose according to their community's values.

The reinforcement learning component shows that dynamic redeployment provides an additional 15% improvement beyond static optimization, pointing toward continuous, demand responsive ambulance positioning.

The estimated impact, 93 to 299 additional cardiac arrest survivors per year, underscores that this is not merely an academic exercise. When minutes mean lives, better algorithms translate directly into lives saved.

HLMI

References

Allen, M., et al. (2021). Developing an OpenAI Gym compatible framework and simulation environment for testing Deep RL agents solving the Ambulance Location Problem. *arXiv:2101.04434*.

Aringhieri, R., Bigharaz, S., Duma, D., and Guastalla, A. (2024). Supporting fair and efficient emergency medical services in a large heterogeneous region. *Journal of Healthcare Informatics Research*, 8(2):400–437.

Bertsimas, D., Farias, V. F., and Trichakis, N. (2011). The price of fairness. *Operations Research*, 59(1):17–31.

Boeing, G. (2025). Modeling and analyzing urban networks and amenities with OSMnx. *Geographical Analysis*, 57(4):567–577.

Chen, Y., Li, Y., Liu, J., and Zhang, Q. (2024). A multi period capacitated facility location problem with backup service for EMS stations. *Computational Urban Science*, 4:143.

Church, R. and ReVelle, C. (1974). The maximal covering location problem. *Papers of the Regional Science Association*, 32(1):101–118.

Crowe, R. P., et al. (2022). Disparities in EMS care delivery in the United States: A scoping review. *Prehospital Emergency Care*, 27(7):830–846.

Daskin, M. S. (1983). A maximum expected covering location model. *Transportation Science*, 17(1):48–70.

Dorsett, M., et al. (2024). Achieving equity in EMS care through quality management systems. *Prehospital Emergency Care*, 28(6):871–881.

Enayati, S., Mayorga, M. E., Toro-Díaz, H., and Albert, L. A. (2023). An analysis of ambulance location problem from an equity perspective. *Socio-Economic Planning Sciences*, 90:101738.

Holmén, J., et al. (2020). Shortening ambulance response time increases survival in OHCA. *Journal of the American Heart Association*, 9(21).

Jagtenberg, C. J. and Mason, A. J. (2020). Improving fairness in ambulance planning by time sharing. *European Journal of Operational Research*, 280(3):1095–1107.

Kim, S., Lee, S., and Jang, H. (2023). Optimizing ambulance allocation in dynamic urban environments. *Applied Sciences*, 13(21):11671.

Liu, Y. and Zeng, B. (2024). Dispatching ambulances using deep reinforcement learning. In *ICLR 2024*.

Liu, K., Li, X., et al. (2020). Ambulance dispatch via deep reinforcement learning. In *ACM SIGSPATIAL 2020*.

Luan, S., et al. (2024). MDP based RL for EMS dispatch through efficiency and fairness. *Transportation Research Record*.

HLMI

Saadatmand, A., et al. (2025). Improving coverage and equity in ambulance planning. *Computers & Industrial Engineering*.

Schulman, J., Wolski, F., Dhariwal, P., Radford, A., and Klimov, O. (2017). Proximal policy optimization algorithms. *arXiv:1707.06347*.

Simore, F., et al. (2025). Ambulance location and relocation under budget constraints. *Health Care Management Science*.

Sivagnanam, A., et al. (2024). Multi agent RL with hierarchical coordination for emergency responder stationing. In *ICML 2024*.

HILMI